Root-Cause Agent — 8D Reports

How NAZAR turns the inspection log into a reason for the rejects and a ready 8D report, offline on the Mac.

---

1. What the agent does

The camera tells us **which** washers are bad. The agent tells us **why**.

While the line runs, the console keeps one row per inspected washer: the camera's verdict (PASS, HOLD or REJECT) plus the sensor readings at the moment of the photo: motor vibration, tunnel light, motor temperature and motor current. When you press "Report", the console sends that log to the agent. The agent then:

1. checks whether the reject rate is stable or jumping (a **p-chart**),
2. splits the washers by machine state ("motor shaking" or "normal") and compares the reject rates (**stratification**),
3. tests whether a difference is real or just luck (**Fisher's exact test**),
4. names the most likely cause and the fix, using what we know about this line,
5. if a fix time was marked, checks **before against after** to prove the fix worked,
6. writes it all up as an **8D report**, with a local AI model wording the explanations.

Steps 1 to 5 are plain arithmetic and always run, in about 1 ms. Step 6 uses the local model and takes about 20 seconds.

```
console log
  -> statistics        p-chart, strata, Fisher's exact test
  -> cause + evidence  which machine state goes with the rejects
  -> 8D template       all facts and numbers
  -> local LLM         rewords the explanations; every number is checked,
                       a section that fails keeps the template text
```

---

2. SPC basics: the p-chart and 3 sigma

SPC (statistical process control) is how factories tell normal ups and downs from a real problem. For good/bad counts the tool is the **p-chart** ("p" for proportion).

The agent cuts the log into windows of 20 washers (more per window on very long logs). For each window it computes **p** = bad washers / washers. It also computes the overall average **$\bar{p}$** over the whole log. If nothing special is happening, each window's p wobbles around $\bar{p}$ by chance, and the size of that wobble is

```
sigma = sqrt( pbar × (1 - pbar) / n )      pbar = overall share flagged
                                           n    = washers in the window
```

Almost all chance wobbles stay within **3 sigma** of the average, so the chart draws an upper control limit UCL = $\bar{p}$ + 3 sigma (and a lower one, never below 0). A window above the UCL is a signal: something real changed.

Real example from the demo log: $\bar{p}$ = 0.127 (13% flagged), n = 20, so sigma = 0.074 and UCL = 0.127 + 3 × 0.074 = **0.35**. The window starting 10:20 had 9 bad out of 20 (p = 0.45), above the UCL, so it is flagged as out of control.

The p-chart tells you **when** things went wrong. It does not tell you **why**. That is the next step.

---

3. Stratification: splitting by machine state

"Stratify" means sort into layers. For each sensor, the agent puts every washer into one of two groups using the console's **amber** limits:

Sensor	Field	"Bad state" group	Default amber
Motor shaking	`vib`	above amber	0.08 g
Tunnel light	`lux`	**below** amber (low light is the problem)	300 lx
Motor temperature	`temp`	above amber	45 °C
Motor current	`ma`	above amber	700 mA

Then it compares the reject rate inside and outside the bad state. From the demo log:

	Washers	Flagged	Rate
Motor shaking above 0.08 g	98	31	32%
Motor normal	202	7	3.5%

The **rate ratio** is 32% / 3.5% = **9.1**: washers were flagged 9 times as often while the motor was shaking. A washer counts as flagged when the camera said REJECT or HOLD. Rows with NO_PART (no washer in the picture) are ignored, never counted as defects. A sensor that was missing in a row (no key in the log) is simply left out for that row.

---

4. Fisher's exact test, in plain words

A big ratio can still be luck if the groups are small. Fisher's exact test answers: **if the machine state had nothing to do with rejects, how likely is a split at least this lopsided?**

Picture the 38 flagged washers as 38 red marbles mixed with 262 white ones. Draw 98 marbles at random (the "shaking" group). Usually you get about 12 red. Getting 31 red by pure chance is astronomically unlikely. Fisher's test counts every possible way the draw could have gone, using exact factorial formulas (the hypergeometric distribution), and adds up the chances of all draws at least as extreme as the one we saw. That total is the **p-value**. Here p ≈ 0.00000000004, written "p < 0.001".

We wrote the test ourselves (about 15 lines in `stats.py`, using log-factorials so it stays fast and accurate) and checked it against textbook values and against a brute-force version with exact integers.

Rules for naming a cause (all must hold):

- at least 5 washers on each side, and at least 3 flagged washers in the bad state,
- the rate ratio is at least 2,
- p is below 0.05 divided by the number of sensors tested (the **Bonferroni** correction: testing 4 sensors gives 4 chances of a fluke, so each one must pass a stricter bar).

If several sensors pass, the one with the smallest p wins. If none pass but one comes close (ratio at least 2, p below 0.1), the report says "**Possible cause ... not proven yet**" and asks for more data. If nothing comes close, the report honestly says **no machine cause was found** and points at the incoming washers. On 40 random "no-cause" demo logs the agent named a machine cause once (2.5%), which is what a 5% false-alarm budget predicts.

---

5. Before and after: proving the fix

When maintenance fixes something, the operator marks the fix time in the console (`fix_at`, with a note). The agent then splits the log at that moment and checks **two** things:

1. **Did the reject rate drop?** Demo: 35 of 219 flagged before (16%), 3 of 81 after (3.7%), Fisher p = 0.003.
2. **Did the cause itself go away?** Demo: 44% of washers were inspected while the motor was shaking before the fix, 2.5% after ($p < 0.001$).

The second check matters: a reject rate can drop by luck, but if the shaking itself has gone, the fix clearly hit the cause. Verdicts: *worked* (rate fell clearly), *cause removed* (the cause state is gone and the rate is lower but not yet provably), *unclear*, *worse*, or *wait* (fewer than 10 washers after the fix).

---

6. The 8D method

8D ("eight disciplines") is the standard problem-solving report in manufacturing, used by car makers and their suppliers. Each "D" is one step:

Step	Name	In NAZAR's report
D1	Team	Operator, maintenance, quality (Team Parakh)
D2	Problem	How many washers, how many flagged, when the p-chart went out of control, the split by machine state
D3	Containment	Stop bad parts escaping now: gate counts, hand-check held washers, stop if the alarm stays amber
D4	Root cause	The cause, the evidence (ratio, p-value) and a **5-Why** chain: ask "why?" five times from the symptom down to the system weakness
D5	Corrective action	The fix, step by step, with who does it
D6	Implement and verify	Before/after numbers and the verdict
D7	Prevent recurrence	Checklists and alarm habits so it does not come back
D8	Close and recognise	When to close the report, and thank the team

What each machine state means on this line, and what to do, lives in `knowledge.py`: shaking means a loose motor mount or belt wobble (tighten the mount, add thread-locker); low light means a dim tunnel LED or dirty diffuser (clean, check the connector, replace); high temperature means an overloaded motor (find the extra load, set belt tension); high current means belt friction or a jam (clear debris, re-track the belt).

---

7. Why a local LLM

A language model writes clear explanations, but sending factory data to a cloud service is a problem: the line may have no internet, the data is the factory's, and the answer must not depend on someone else's server. So NAZAR runs the model **on the Mac itself**, fully offline.

- **Model: Llama 3.1 8B Instruct** (Meta). "8B" = 8 billion parameters, large enough to write good plain English, small enough for a laptop-class computer.
- **4-bit**: each parameter is stored in 4 bits instead of 16. The model shrinks from about 16 GB to 4.2 GB and fits in the Mac's memory next to the camera AI, with a small loss in quality.
- **Apple MLX**: Apple's machine-learning library for M-series chips. The CPU and GPU share the same memory, so the model does not have to be copied to a separate graphics card. On the M4 it loads in about 7 s and writes about 16 tokens (word pieces) per second, including reading the prompt.
- **One worker thread**: MLX must load and run the model on the same thread, so the service keeps one dedicated model thread and the web requests only hand it jobs. One report is written at a time; a second request while it is busy gets the template report straight away.

8. Facts come from templates; the LLM only explains

An earlier 8B version, when allowed to write the whole report, invented "the gate malfunctioned". A report that invents facts is worse than no report. So the work is split:

- **Facts** (title, summary, D1, D2, D3, D4 evidence, D6, every number): filled in from the statistics by fixed templates. The LLM never touches them.
- **Explanations** (the five whys, D5, D7, D8): the template already has a good version; the LLM may reword it for the reader.

Every section the LLM writes goes through checks before it is used:

Check	Catches
Every number must already appear in the facts or the template	invented numbers ("12 Nm", "22%")
No blame on the camera, AI, gate or sensors	"the gate malfunctioned"
No doubled words or repeated sentences	small-model loops ("8D 8D ...")
Each 5-Why step must still be a why-question with an answer	steps that lost their meaning
Right language, sensible length	stray other-language or runaway text

A section that fails keeps the template text. If fewer than 4 of the 8 sections pass, the whole report uses the template. The response says exactly which sections the LLM wrote (`by` on each section, and `llm.accepted` / `llm.rejected`). With `use_llm:false` the report is template-only and takes about 1 ms. The loop and why-form checks were added after a test run produced a 5-Why step that had turned into a yes/no question and a closing section that repeated itself. In the final English run on the no-cause log, the checks rejected one 5-Why step, which kept its template text.

9. Language

The report is in **English**, and `lang` defaults to `"en"`. Hindi is planned / template only, not shown in the demo: hand-written Hindi fact templates exist in the code and pass the unit tests without the LLM, but the Hindi LLM path is not part of the demo.

10. The HTTP API

The agent is a small web service at `http://127.0.0.1:8766` (only this Mac can reach it), as written in `CONTRACT.md` section 5. It uses only Python's standard library.

Endpoint	Returns
`GET /health`	`{"ok":true,"llm":"installed"/"loaded"/"missing","busy":false,"version":"2.0.0"}`
`POST /report`	the log in, statistics + 8D report out

Request (shortened): `{"lang":"en","use_llm":true,"limits":{...},"fix_at":...,"fix_note":"...","inspections":[{"at":...,"verdict":"REJECT","vib":0.11,"lux":805,"temp":36.1,"ma":330}, ...]}`.

Response: `stats` (n, bad, rate, `pchart` windows, `strata`, `cause`, `before_after`) and `report` (title, summary, sections D1 to D8). Fewer than 20 inspections gives `enough:false` with a message and no report. Every answer allows any origin (CORS), `OPTIONS` is answered, and a bad request never crashes the service: it answers `{"ok":false,"error":"..."}`.

---

11. How to run and demo

```
cd nazar-agent
./run.sh                    # or double-click "Start Agent.command"
python3 tools/make_demo_log.py --story loose-mount -o demo/loose-mount.json
curl -s -X POST -H 'Content-Type: application/json' \
     --data @demo/loose-mount.json http://127.0.0.1:8766/report
```

`run.sh` uses the Python in `10-AI/.venv` (it has `mlx-lm`) and the model in `10-AI/llm`; both can be changed with `NAZAR_PY` and `NAZAR_LLM_DIR`. The model loads in the background at start, so start the agent a minute before the demo.

Demo stories: `loose-mount` (shaking episodes, then a fix), `no-cause` (random rejects), `dim-light` (low tunnel light), `few` (12 rows). Suggested live demo: run the line with the motor mount slightly loose, mark the fix in the console after tightening it, then press Report.

---

12. Measured timings (M4 Mac, English, demo log of 300 washers)

Case	Time
`use_llm:false` (template only)	about 1 ms
First report after start (model load 6.9 s + writing 20.1 s)	27.0 s
Later reports, loose-mount log (320 tokens)	19.7 to 20.2 s
Later report, no-cause log (239 tokens)	15.1 s
Unit tests (16)	under 0.1 s

The same log gives the same text every time (fixed random seed).

---

13. Real example output (loose-mount demo, LLM on)

Title: Rejects rise when the motor shakes

Summary (template): 300 washers inspected, 38 flagged (13%). Washers were flagged 9.1 times as often while the motor was shaking above its amber limit ($p < 0.001$). Most likely cause: a loose motor mount or a wobbling belt. After the fix the flagged rate went from 16% to 3.7%.

D6 (template): Fix marked at 10:37: tightened motor mount screws, added thread-locker. Before the fix, 44% of washers were inspected in the motor shaking amber state; after it, 2.5%. Flagged washers: 35 of 219 before (16%), 3 of 81 after (3.7%). The flagged rate fell clearly (Fisher's exact test, p = 0.003): the fix worked.

5-Why, steps 3 to 5 (LLM, checked):

3. Why is the motor shaking? The motor is shaking because its mount has worked loose, or the belt is wobbling on the rollers.
4. Why did the mount work loose? The mount worked loose because constant vibration slowly backed off screws that had no thread-locker or spring washer.
5. Why was it not caught earlier? It was not caught earlier because the motor mount is not on a regular checklist and the amber shaking alarm was not acted on.

D5 (LLM, checked): Stop the belt, tighten the motor mount screws and add thread-locker or spring washers. The operator stops the belt and acts on alarms, maintenance does the repair, and quality checks the result.

---

14. What is not verified yet

- **Real data.** All tests use generated demo logs. The agent has not yet been run on a log from the real line or called by the real console.
- **Meaning drift.** The checks catch invented numbers, blame, loops and broken 5-Why steps, but not a sentence that is merely less accurate. In the no-cause run the LLM's D5 said "maintenance will repair the current batch" and dropped the hand-sample step. Read the LLM sections before presenting; the template version is always available with `use_llm:false`.
- **Wording.** The LLM's 5-Why step 2 in the example above is clumsy ("a tilting and shifting belt tilts and shifts washers"). Correct, but not elegant.
- **Confounded sensors.** If two sensors always go amber together (for example shaking and high current), the statistics cannot tell which one is the cause; the agent picks the stronger link and lists the other.
- **Busy model.** A second report requested while one is being written gets the template version; this was designed in but not load-tested.
- **Hindi** is planned / template only and not part of the demo.